

A Hybrid Elgamal – Paillier Approach for Achieving Practical Fully Homomorphic Encryption in Cloud Computing

Krishna Agarwal¹, Aditi Srivastava², Mahesh Kumar Tiwari³, Rinku Raheja⁴
Dr. Shalini Lamba⁵, Amit Srivastava⁶

^{1,2}Research Scholar, National PG College, Lucknow University
^{3,4,5,6} Assistant Professor, National PG College, Lucknow University

Received: Jul 18, 2025

Accepted: Aug 18, 2025

Published: Aug 26, 2025

Abstract:

Data in the processing stage becomes vulnerable and unsecure as one cannot fully trust the individual responsible for the operation. This research paper aims to offer privacy for data in processing using a method known as Homomorphic Encryption. We have examined all significant and influential cryptographic algorithms, including RSA, Paillier, Elgamal, and the Goldwasser-Micali algorithm, which can be applied to data that is currently in use. A code is written in python and is executed to conduct a brief analysis of functioning and efficiency of each algorithm. A comparison chart has been created based on the time taken by each algorithm in seconds, along with the respective encryption time, homomorphic encryption time, and decryption time. Additionally, a line graph has been generated to evaluate the performance of each algorithm. The major part of our research is an attempt to combine two algorithms i.e. Paillier cryptography algorithm and Elgamal cryptography algorithm and have created a hybrid approach which will not only provide the fully homomorphic capability but also enhance the security features for the computation of in use data. This research could have a significant impact as this technology is increasingly required in the IT industry, given the rise in data, necessitating enhanced security features.

Keywords:

Cloud Computing, Partial Homomorphic Encryption, RSA, Paillier, ElGamal.

Introduction:

Cloud Computing refers to the use of servers remotely in order to reduce security management task, risk analysis in place of local servers. It is primarily aimed to resolve the problem of multiple physical servers with the help of virtualization. This reduction in number of physical servers not only helps in cost estimation but also provides various features such as scalability, accessibility, pay per use, collaboration and backup and disaster recovery. The cloud computing technology works on the concept of resource pooling i.e. various resources such as processors and storage are shared collectively and users are allocated these resources. The most interesting feature of cloud computing is that users can get access to any specific resources anytime anywhere without any human intervention. Further, cloud computing supports elasticity i.e. it can allocate and deallocate resources to align with the demand.

In the world of emerging technologies, the major concern that is rising is the safety of the data. To provide the protection to the user data the concept of encryption came into existence. The encryption refers to the conversion of message(plain text) to the encoded form of text known as cipher text. Thus an encryption becomes an important method to avert data breaches, uphold customer trust and guarantee compliance with regulations. Encryption helps in safeguarding against financial setbacks, legal issues and damage of reputation of individuals and companies.

Traditional encryption methods involved the computations to be performed by the cloud service providers on the ciphertext by decrypting it and then performing the computations. This raised the security concerns among the shareholders. Thus, the concept of homomorphic encryption emerged where computations on encrypted data are made possible.

Literature Review:

Name of Author and year of publication	Objective	Encryption Scheme used	Conclusion	Research Gap
Abbas Acar, 2018	To give a clear information of different homomorphic encryption techniques.	HE and FHE	Performance of any encryption technique is dependent on speed, security and simplicity.	Fully Homomorphic Encryption needs more attention.
Sefik Ilkin Serengil, 2024	Development of a lightweight and hybrid PHE secure framework application which provide more efficient and practical solution	LightPHE	Provides a single interface where users can use multiple PHE algorithm at one place which becomes an efficient tool for developer.	Efficiency and scalability can be increased and more optimization can be achieved.
Weinan Liu, 2025	A study between PHE and Approximate HE for analyzing bootstrapping, variants and improved precisions.	PHE and AHE	A comprehensive and deep analysis of Approximate HE have been done on the basis of survey.	Improvement required in AHE scheme in the field of Machine Learning.
Joy Ezinwanneamaka, 2024	Optimized HE algorithms are used to reduce computational overhead.	RSA, AES and HE	Batching techniques and levelled HE can be use to reduce overhead and computational cost.	Enhancement of usability and efficiency in cloud application.
Bhawna Parihar, 2025	To propose a secure Homomorphic based secure computational framework with which network and hardware efficiency of distributed computer could be increased.	Multi-Part Computation and HE	Computational efficiency, scalability, bootstrapping overhead and security was improved of the traditional HE algorithms.	Further optimization could be done in this hybrid framework so that it can be applied in real world.
Muna Mohammed Saeed Altee	To secure banking data in cloud by	RSA and Paillier	Time and cost of performing	Many more sectors other than banking

	using HE cryptographic algorithms.		operations on banking data has been reduced.	should have been explored.
V. Biksham, 2017	To maintain the confidentiality and privacy of cloud data.	PHE and FHE	The problem identification has been discussed and a brief overview of Homomorphic encryption has been given.	To reduce computational overhead of Fully Homomorphic encryption .
Kamal Kumar Chauhan, 2015	To secure data which is in processing state by using various homomorphic encryption techniques.	Pailler and RSA	Traditional encryption techniques lacks security issues which can be overcome using homomorphic encryption.	More feasible HE algorithms should be designed which can be implemented easily in real world

Cryptography and its importance-:

Cryptography is a technique which helps in maintaining the security of the data by preventing it from any unauthorized access. It ensures the confidentiality and integrity of data by converting the plain text to encoded text. Encoding and Decoding is done using complex mathematical set of rules which becomes difficult for the third person to understand and make changes.

Types of Cryptography –

On the basis of number of keys involved in the successful transfer of ciphertext, cryptography is mainly classified into two types: symmetric-key cryptography and asymmetric-key cryptography.

Symmetric-Key Cryptography – Symmetric Key Cryptography, generally referred as private key cryptography in simple terms can be understood as “My lock, My key”. In this system a single key is used for both encryption and decryption. The sender encrypts the message using a key before sending it to the receiver and sends that key along with the message. Thus this way ciphertext gets protected from potential cyber threats. The receiver can only decrypt the message using the private key with which the text was encrypted. But it is just like “hanging the key on the door after locking the door” i.e. this demands the secure transfer of private key over the network. Thus, the concept of symmetric key cryptography is vulnerable to security threats as the private key is not only made available to the receiver but also to the attackers making it inefficient for encryption. Data Encryption Standard (DES) and Advanced Encryption Standard (AES) comes under the commonly used symmetric cryptography.

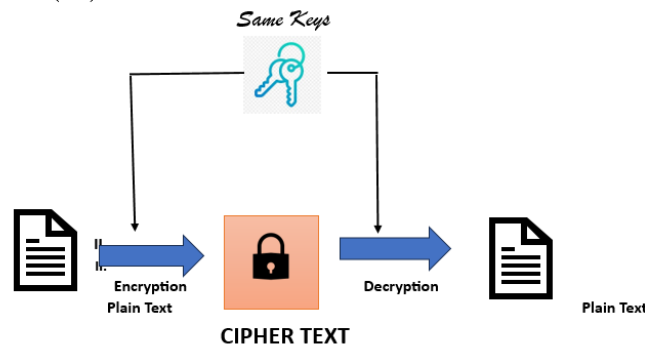


Fig 1: Symmetric Key Cryptography

Asymmetric-Key Cryptography – In symmetric key cryptography, it is seen that the transfer of key used for decrypting the ciphertext over the network makes the ciphertext vulnerable to the middleman thus making the unsecure transfer. To eliminate this vulnerability and to enhance the security the concept of asymmetric key cryptography or public key cryptography is introduced which works on two keys – one for encryption and the other for decryption. Typically, the plaintext if encrypted through the public key can be decrypted using the private key of same node and if encrypted using the private key can be decrypted using the public key of same node. There are four possibilities which can be made for encryption.

Table 1: Possible Combinations for Encryption/Decryption

Encrypted using	Can be Decrypted using			
	Public Key of Node A	Private Key of Node A	Public Key of Node B	Private Key of Node B
Public Key of Node A	✗	✓	✗	✗
Private Key of Node A	✓	✗	✗	✗
Public Key of Node B	✗	✗	✗	✓
Private Key of Node B	✗	✗	✓	✗

Selection of Cases –

Case 1- The case is similar to symmetric cryptography where the private key is transferred over the network making it unsecure.

Case 2 – Every node has access to public key of node A, so ciphertext can be decrypted by anyone making it vulnerable to attackers.

Case 3 – The encrypted text can only be decrypted by the receiver node as it needs private key of receiver. Thus, it is a secure method.

Case 4 - Every node has access to public key of node B, so ciphertext can be decrypted by anyone making it vulnerable to attackers.

Thus Case 3 i.e. encryption using public key of receiver node and decryption using private key of receiver node is used in case of asymmetric key cryptography. A secure and safe environment is created for data transfer without the need of key sharing. The implementation of asymmetric key cryptography is facilitated using the Digital Signatures.

In asymmetric key cryptography, unauthentic sources may use the public key of receiver's node to send the illegitimate data. Thus, digital signature serves the purpose of verification of authenticity of the source of ciphertext in an opposite manner to that of asymmetric key cryptography. The Case 2 is used in the digital signatures where the source is validated by the receiver by using the public key of source node. Digital Signature Algorithm (DSA) is among the most commonly and widely used techniques for digital signatures. Other methods include Diffie-Hellman and Elliptic curve cryptography.

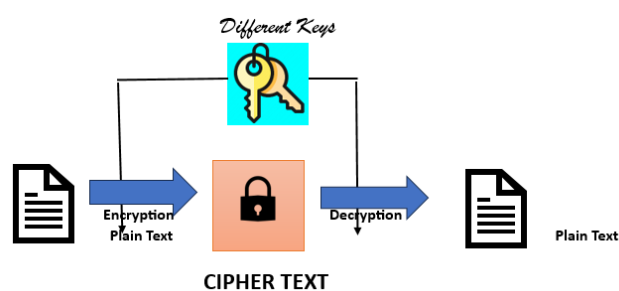


Fig 2: Asymmetric Key Cryptography

Homomorphic Encryption:

It is an encryption technique which starts the computations on the data without decrypting it i.e. computations are performed on the cipher text itself. In traditional approach of encryption, message needs to be decrypted first to perform the computations. This traditional method lacks the safety of confidential data, so in order to prevent data from being vulnerable to other person the concept of homomorphic encryption came into use. The homomorphic encryption is widely used in every field. In healthcare systems, the data of patients is uploaded on cloud servers where privacy is a major concern. Data analysis and machine learning model training also uses the concept of homomorphic encryption. Consider two messages M_1 and M_2 and their encrypted texts as $E(M_1)$ and $E(M_2)$. As per the property of homomorphic encryption,

$E(M_1) + E(M_2) = E(M_1 + M_2)$ – Additive Homomorphic Encryption

$E(M_1) * E(M_2) = E(M_1 * M_2)$ – Multiplicative Homomorphic Encryption

RSA scheme is used for Multiplicative Homomorphic encryption and Paillier cryptosystem is used for Additive Homomorphic encryption.

There are three basic types of homomorphic encryption techniques. They are:

Partial Homomorphic encryption – Partial homomorphic encryption involves only a single type of operation. It works for either addition (and subtraction) operations or multiplication (and division) operations.

Fully Homomorphic encryption – It involves both types of operations on the cipher text i.e. additive operations and multiplicative operations. In addition, it also supports logical operations AND, OR and NOT. However, it is computationally extensive.

Somewhat Homomorphic encryption – The feature of this type of encryption technique is almost similar to fully homomorphic encryption but the drawback is it supports only few number of operations.

Applications of Homomorphic Encryption

Being very valuable technique for security of sensitive data we following application of homomorphic encryption:

1. Security in Cloud Computing- The feature of providing data privacy and Secure Multi-Party Computation (SMPC) makes HE the best technique for cloud computing. It enables to hide the underlying information from the person who is performing operation on the data since the processing is performed on encrypted data itself. It is useful when an unauthorized authority has to perform processing on personal and corporate data. In this way it helps in data privacy. Also when multiple organization want to analyze the group of data but do not want to share with each other openly then they can take benefit of homomorphic encryption which we call SPMC.
2. Healthcare sector - Medical data being the most sensitive one data has to be protected. When medical record needs to be given to researchers or to the diagnosis center then the personal information should be hided so that the analysis could be performed by multiple people without leakage of data. In this way privacy-preserving medical data analysis could be performed and also the diagnosis over the cloud.
3. Privacy in Machine Learning- We know that the models of machine are trained using raw facts and figures. If these raw facts are provided in encrypted form, then our data will remain safe from the party involved during training of the model. Therefore, the prediction or the result provided by that machine will also be in encrypted form. In this way homomorphic encryption helps in machine learning.
4. Secure Voting System- In the era of electronic voting system, homomorphic encryption can prevent the tampering of data and can maintain the privacy by tallying the votes in encrypted form only. In this way the votes can also be matched and the data will remain hidden from the operator.
5. Multiple Data Analysis- Many organization wants to collaborate and divide their task with other organization but they find they lack the security issues and think their data will be at risk if given to some other party. But due to homomorphic encryption now they can collaborate easily without the risk of data to be misused.

RSA algorithm

RSA which stands for Rivest – Shamir – Aldeman is a scheme of asymmetric key cryptography. Consider the plain message M to be encrypted. Two large numbers (for security purposes) between p and q are selected and their product is obtained. Following this, Euler's Toitient function (Φ_n) is obtained which is $(p-1)*(q-1)$. A value of e is chosen such that $1 < e < \Phi_n$ and greatest common divisor of Φ_n and e is 1. The value of d is calculated which is $e^{-1} \bmod \Phi_n$. Finally $\{e, n\}$ becomes the public key and $\{d, n\}$ becomes the private key. The encrypted message CT_1 becomes $M^e \bmod n$. For decryption $C^d \bmod n$ is calculated. It is observed that value of e is used in the process of encryption while the value of d is used in the process of decryption which follows the property that public key ($\{e, n\}$) for encryption and private key ($\{d, n\}$) for the decryption process.

Homomorphic property in RSA – The RSA encryption scheme supports the multiplicative homomorphism. Consider two plain texts t_1 and t_2 . The ciphertexts for the respective messages are obtained which are $t_1^e \bmod n$ and $t_2^e \bmod n$ respectively. $c_1 \cdot c_2 = (t_1^e \cdot t_2^e) \bmod n = (t_1 \cdot t_2)^e \bmod n$. Thus, as per the multiplicative property of homomorphic encryption the product of the plaintexts must be equal to the product of ciphertexts which is clearly followed by the RSA scheme. This property supports the partial homomorphic encryption. However, it does not

support additive operations on the encrypted text making it unsuitable for fully homomorphic encryption.

DSA algorithm and Digital Signature

Digital Signature Algorithm abbreviated as DSA is a asymmetric cryptography technique in which public key is used for encryption and private key is used for decryption of the message. The concept of hash function makes it more secure and one of the best algorithm for transfer of data over the internet. In this the message M is first passed through the hash function where the hash code is generated and then the message is encrypted using the public key. Now message along with the hash code is transferred further towards the receivers end. Once the message is reached it is decrypted using the private key and here only the hash code is generated again which is compared with the hash code generated at the time of encryption. This comparison is called signature verification step. If match is successful then the message is received otherwise further action is taken. Message authentication and message integration are the two services provided by the DSA algorithm. It is to be noted that confidentiality is not provided by this algorithm. DSA which stands for Digital Signature Algorithm is a cryptographic algorithm designed for generation and verification of digital signatures to ensure the authenticity of data and the source of data, enhancing security. Consider the plain message M to be encrypted. Two prime numbers p and q and the generator of a subgroup g are selected. A random integer x behaves as the primary key. The public key y is calculated as $g^x \bmod p$. The plain message is passed through a hash function to obtain the message digest $H(m)$. A random number k is chosen such that $0 < k < q$. r and s are calculated as $(g^k \bmod p) \bmod q$ and $k^{-1}(H(m) + xr) \bmod q$ respectively. (r, s) becomes the generated signature. For the process of verification, the conditions $0 < r < q$ and $0 < s < q$ are verified. Then, the following computations are done –

$$w = s^{-1} \bmod q$$

$$u_1 = (H(m) \cdot w) \bmod q$$

$$u_2 = (r \cdot w) \bmod q$$

$$v = ((g^{u_1} \cdot y^{u_2}) \bmod p) \bmod q$$

If $v=r$ is satisfied, the signature is valid else it is invalid.

With the help of mathematical calculations it is clearly seen that for the digital signature generation, private key is used and for verification process, public key is used.

Paillier Cryptosystem:

Paillier cryptosystem is an asymmetric encryption algorithm that follows the additive homomorphism. Unlike other algorithms, it is a non-deterministic algorithm i.e. it gives different results for same set of inputs at different time periods. Two large numbers p and q are chosen and $n=p \cdot q$ is computed. The value of λ is computed i.e. lcm of $p-1$ and $q-1$. A random integer g is selected such that g belongs to $\mathbb{Z}_{n^2}^*$. The value of μ is computed by the formula $(L(g^\lambda \bmod n^2))^{-1} \bmod n$ where L is a function defined as $L(x) = (x-1)/n$. Finally, the (n, g) becomes the public key and the (λ, μ) becomes the private key. Consider a plain message m to be encrypted where $0 \leq m < n$. A random number r is chosen such that $\gcd(r, n) = 1$. $g^m \cdot r^n \bmod n^2$ behaves as the ciphertext. For the decryption process of ciphertext c , $L(c^\lambda \bmod n^2) \cdot \mu \bmod n$ is the plain message.

Homomorphic property in Paillier Cryptosystem – Consider two plain messages m_1 and m_2 , then $D(E(m_1, r_1) \cdot (E(m_2, r_2) \bmod n^2)) = m_1 + m_2 \bmod n$

Elgamal Algorithm:

Elgamal Algorithm is a Diffie Hellman key exchange based public key cryptography algorithm that is designed for digital signatures and enhancements in the security in communication. It is multiplicative homomorphic in nature. The algorithm is generally divided in three phases – key generation, encryption and decryption. In the key generation phase, a random large prime number p is selected and a private key d is selected. A first part e_1 of public key is selected at random and the second part of public key is calculated as $e_1^d \bmod p$. (e_1, e_2, p) becomes the public key. For the encryption, a random number r is chosen and c_1 and c_2 are calculated as $e_1^r \bmod p$ and $(\text{plaintext} \cdot e_2^r) \bmod p$ and (c_1, c_2) behaves as the ciphertext. For the decryption process, the value of $[c_2 \cdot ((c_1)^d)^{-1}] \bmod p$ is calculated.

Homomorphic property in Elgamal Algorithm -: Consider two ciphertexts as

$$C_1 = (c_{11}, c_{12}) = (g^{k_1} \bmod p, m_1 \cdot y^{k_1} \bmod p)$$

$$C_2 = (c_{21}, c_{22}) = (g^{k_2} \bmod p, m_2 \cdot y^{k_2} \bmod p)$$

On multiplication, $C' = (c_{11} \cdot c_{21} \bmod p, c_{12} \cdot c_{22} \bmod p)$

Decrypted ciphertext: $m' = m_1 \cdot m_2 \bmod p$

Goldwasser-Micali cryptosystem Algorithm:

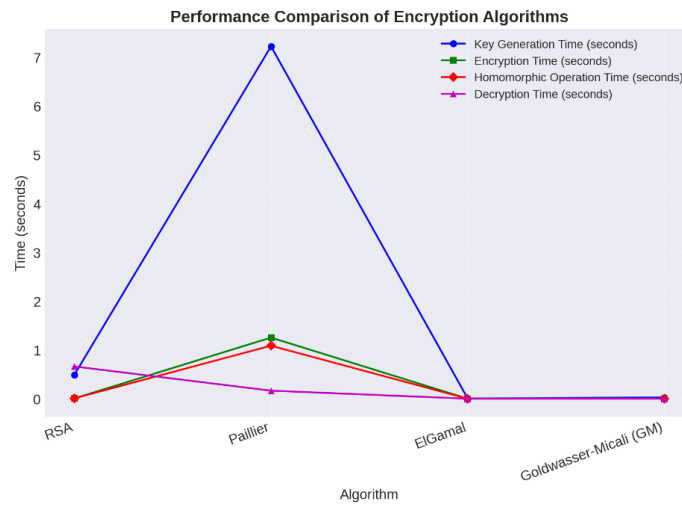
It is a unique type of probabilistic and XOR multiplicative homomorphic encryption that processes one bit at a time, which can make it impractical for large data sets. However, its significance lies in the probabilistic nature of the encryption, enhancing its security. During the key generation process, two unique large prime numbers, p and q , are chosen with the condition that $p \% 4 = 3$ and $q \% 4 = 3$. The product N is calculated as $p \cdot q$. Additionally, a non-residue a is determined so that $ap(p-1)/2 = -1 \pmod{p}$ and $aq(q-1)/2 = -1 \pmod{q}$. The combination (a, N) forms the public key, while (p, q) constitutes the private key. In the encryption stage, for each bit m_i , a random value b_i is selected from the group of units modulo N , ensuring that $\gcd(b_i, N) = 1$. The ciphertext is then generated as $c_i = b_i^2 \cdot a^{m_i} \pmod{N}$. For decrypting $(c_1, c_2, \dots, c_n)$, prime factorization (p, q) is utilized to ascertain if each c_i is a quadratic residue; if it is, then $m_i = 0$; if not, $m_i = 1$.

Table 2: Comparative study of Partially Homomorphic Algorithms

Algorithm	RSA	Paillier	ElGamal	Goldwasser-Micali
Type	Multiplicative	Additive	Multiplicative	XOR – based
Homomorphic Property	$E(M_1) \times E(M_2) = E(M_1 \times M_2)$ $E(M_1) \times E(M_2) = E(M_1 \times M_2)$ $E(M_1) \times E(M_2) = E(M_1 \times M_2)$	$E(M_1) + E(M_2) = E(M_1 + M_2)$ $E(M_1) + E(M_2) = E(M_1 + M_2)$ $E(M_1) + E(M_2) = E(M_1 + M_2)$	$E(M_1) \times E(M_2) = E(M_1 \times M_2)$ $E(M_1) \times E(M_2) = E(M_1 \times M_2)$ $E(M_1) \times E(M_2) = E(M_1 \times M_2)$	Probabilistic encryption utilizing bitwise XOR
Security Basis	Factoring large primes	Hardness of composite residue class problem	Discrete logarithm problem	Quadratic non-residues
Efficiency	Moderate (Key generation requires significant time)	High (Facilitates multiple additions)	Slower (Involves exponentiation)	Inefficient (Encrypts a single bit at a time)
Use Cases	Secure transactions, digital signatures	Secure cloud computing, e-voting	Secure messaging, authentication	Secure communication, pseudorandom generators

Table 3: Comparative Analysis of Partially Homomorphic Algorithms

Algorithm	Key Generation Time (seconds)	Encryption Time (seconds)	Homomorphic Operation Time (seconds)	Decryption Time (seconds)
RSA	0.489967	0.005689	0.007661	0.661253
Paillier	7.225038	1.253436	1.086937	0.164780
ElGamal	0.005133	0.000050	0.000002	0.000023
Goldwasser-Micali (GM)	0.025653	0.000012	0.000002	0.000098

**Fig 3: Comparative analysis on the basis of time required in performing different operations****Table 4: Comparative Analysis of Partially Homomorphic Algorithm on basis of their speed**

Algorithm	Key Generation Time (seconds)	Encryption Time (seconds)	Homomorphic Operation Time (seconds)	Decryption Time (seconds)
RSA	Moderate	Fast	Moderate	Fast
Paillier	High	Moderate	Fast	Moderate
ElGamal	High	Slow	Slow	Moderate
Goldwasser-Micali (GM)	Very High	Very Slow	Very Slow	Moderate

Proposed Work: The partially homomorphic cryptographic algorithms have disadvantage that they are limited to single type of operations on the ciphertext - either additive operation or multiplicative operation. Though the disadvantage has significantly been overcome by the use of fully homomorphic encryption algorithms, but it has computational overhead and is impractical for many cases. A hybrid approach of two partially homomorphic algorithms is proposed as the solution for this problem. The algorithms chosen for the hybrid approach are Paillier cryptography algorithm and Elgamal cryptography algorithm due to the following reasons –

- Paillier algorithm promotes additive homomorphism whereas ElGamal algorithm promotes multiplicative homomorphism
- Fast and efficient homomorphic encryption
- Secured formation of algorithm that is less vulnerable to quantum attacks as compared to other algorithms

Table 5: Algorithms used in Hybrid Approach

Algorithm	Homomorphic Property	Principle
ElGamal	Multiplicative	Discrete Logarithm Problem
Paillier	Additive	Composite Residue Class Problem

Introduction to Hybrid Homomorphic Algorithm:

The hybrid approach works on the dual encryption model i.e. it works on the two layers i.e. the plaintext is first encrypted using the Paillier cryptography algorithm for attaining additive homomorphism and then through the Elgamal cryptography algorithm for multiplicative homomorphism.

Steps used in the hybrid approach –

Key Generation -

1. Paillier Algorithm setup
 - a. Select two large prime numbers p and q
 - b. Calculate $n = p \cdot q$, and $g = n + 1$
 - c. Compute $\lambda = \text{lcm}(p-1, q-1)$ and $\mu = \lambda^{-1} \bmod n$
 - d. Private Key = (n, g)
 - e. Public Key = (λ, μ)
2. Elgamal Algorithm setup
 - a. Select a large prime number P and generator g
 - b. Select private key x and calculate $y = g^x \bmod P$
 - c. Private Key: (P, g, y)
 - d. Public Key: x

Encryption –

1. Paillier Encryption

$$C_p = E_P(m) = g^m \cdot r^n \bmod n^2$$
2. Elgamal Encryption

$$C_e = (g^k \bmod P, m \cdot y^k \bmod P)$$

Homomorphic Operations -

1. **Homomorphic Addition (Using Paillier)**

$$E_P(m_1) \cdot E_P(m_2) = E_P(m_1 + m_2)$$
2. **Homomorphic Multiplication (Using ElGamal)**

$$E_E(m_1) \cdot E_E(m_2) = E_E(m_1 * m_2)$$

Ciphertext Conversion (CTL Function) -

A transformative function is defined to switch from Paillier to ElGamal:

$$T(C_p) = C_p \bmod P$$

This makes sure that Paillier ciphertext is valid within the ElGamal field.

Decryption Process -1. **ElGamal decryption:**

$$M' = C_2 \cdot (C_1^x)^{-1} \bmod P$$

2. **Paillier decryption:**

$$M = (C^\lambda \bmod n^2 - 1) / n \times \mu \bmod n$$

Output of implementation of algorithm -

Initial Value: 42

Stage 1 Output: 183279481927419827491 (Example)

Stage 2 Output: (71263712637126, 92837492837498374)

Intermediate Recovery: 183279481927419827491

Final Output: 42

Secure Process Completed Successfully!

Table 6: Comparison Table of Paillier, ElGamal and Hybrid Approach of Cryptography

Feature	Paillier	ElGamal	Hybrid (Paillier + ElGamal)
Homomorphic Addition	Yes	No	Yes
Homomorphic Multiplication	No	Yes	Yes
Encryption Speed	Fast	Fast	Slow
Decryption Speed	Fast	Fast	Slow
Security Strength	Moderate	Moderate	Stronger Together
Quantum Resistant	No	No	Extendable with Lattice Crypto

Conclusion:

The study concludes that the hybrid approach of the ElGamal and Paillier algorithms have significantly tackled the mathematical complexities and computational overheads involved in the Fully Homomorphic Encryption techniques. The hybrid approach of the ElGamal and Paillier algorithms would be a great innovation in the field of cybersecurity in the current generation where the data is the most important concern. Not only eliminating the computational complexities, it becomes less vulnerable to quantum attacks as compared to the other partially homomorphic algorithms.

Though the computational overhead is reduced, the future work can be done to tackle the time complexities increased by the combination of algorithms.

REFERENCES:

1. Acar, Abbas, et al. "A survey on homomorphic encryption schemes: Theory and implementation." *ACM Computing Surveys (Csur)* 51.4 (2018): 1-35.
2. Tebaa, Maha, Saïd El Hajji, and Abdellatif El Ghazi. "Homomorphic encryption method applied to Cloud Computing." *2012 National Days of Network Security and Systems*. IEEE, 2012.
3. Geng, Yang. "Homomorphic encryption technology for cloud computing." *Procedia Computer Science* 154 (2019): 73-83.
4. Chinnasamy, P., et al. "Efficient data security using hybrid cryptography on cloud computing." *Inventive Communication and Computational Technologies: Proceedings of ICICCT 2020*. Springer Singapore, 2021.
5. Rakhra, Manik, et al. "Hybrid Cryptography in Cloud Computing." *2024 11th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions)(ICRITO)*. IEEE, 2024.
6. Chatterjee, Rishav, Sharmistha Roy, and U. G. Scholar. "Cryptography in cloud computing: a basic approach to ensure security in cloud." *International Journal of Engineering Science* 11818 (2017).
7. Alloghani, Mohamed, et al. "A systematic review on the status and progress of homomorphic encryption technologies." *Journal of Information Security and Applications* 48 (2019): 102362.
8. Jaber, Aws Naser, and Mohamad Fadli Bin Zolkipli. "Use of cryptography in cloud computing." *2013 IEEE International conference on control system, computing and Engineering*. IEEE, 2013.
9. Marcolla, Chiara, et al. "Survey on fully homomorphic encryption, theory, and applications." *Proceedings of the IEEE* 110.10 (2022): 1572-1609.
10. Alaya, Bechir, Lamri Laouamer, and Nihel Msilini. "Homomorphic encryption systems statement: Trends and challenges." *Computer Science Review* 36 (2020): 100235.
11. Chase, Melissa, et al. "Security of homomorphic encryption." *HomomorphicEncryption.org*, Redmond WA, Tech. Rep (2017).
12. Doröz, Yarkin, Erdinç Öztürk, and Berk Sunar. "Accelerating fully homomorphic encryption in hardware." *IEEE Transactions on Computers* 64.6 (2014): 1509-1521.
13. Sasikumar, K., and Sivakumar Nagarajan. "Comprehensive review and analysis of cryptography techniques in cloud computing." *IEEE Access* (2024).
14. Han, Kyoohyung, and Dohyeong Ki. "Better bootstrapping for approximate homomorphic encryption." *Cryptographers' Track at the RSA Conference*. Cham: Springer International Publishing, 2020.
15. Chachapara, Kajal, and Sunny Bhadlawala. "Secure sharing with cryptography in cloud computing." *2013 Nirma University International Conference on Engineering (NUiCONE)*. IEEE, 2013.
16. Pulido-Gaytan, Bernardo, et al. "Privacy-preserving neural networks with homomorphic encryption: C hallenges and opportunities." *Peer-to-Peer Networking and Applications* 14.3 (2021): 1666-1691.
17. Wang, Wei, et al. "Accelerating fully homomorphic encryption using GPU." *2012 IEEE conference on high performance extreme computing*. IEEE, 2012.
18. Halevi, Shai, and Victor Shoup. "Design and implementation of HELib: a homomorphic encryption library." *Cryptology ePrint Archive* (2020).
19. Han, Kyoohyung, et al. "Logistic regression on homomorphic encrypted data at scale." *Proceedings of the AAAI conference on artificial intelligence*. Vol. 33. No. 01. 2019.
20. Cheon, Jung Hee, et al. "Introduction to homomorphic encryption and schemes." *Protecting Privacy through Homomorphic Encryption* (2021): 3-28.
21. Rakhra, Manik, et al. "Hybrid Cryptography in Cloud Computing." *2024 11th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions)(ICRITO)*. IEEE, 2024.
22. Liu, W., You, L., Shao, Y., Shen, X., Hu, G., Shi, J., & Gao, S. (2025). From accuracy to approximation: A survey on approximate homomorphic encryption and its applications. *Computer Science Review*, 55, 100689.
23. Serengil, S. I., & Ozpınar, A. (2024). LightPHE: Integrating Partially Homomorphic Encryption into Python with Extensive Cloud Environment Evaluations. *arXiv preprint arXiv:2408.05219*.
24. Ike, J. E., Kessie, J. D., Popoola, R., Azeez, M. A., & Onibokun, T. (2024). A Novel Approach to Cloud Data Encryption using Homomorphic Encryption.
25. Biksham, V., & Vasumathi, D. (2017). Homomorphic encryption techniques for securing data in cloud computing: A survey. *International Journal of Computer Applications*, 975(8887).
26. Chauhan, K. K., Sanger, A. K., & Verma, A. (2015, December). Homomorphic encryption for data security in cloud computing. In *2015 International conference on information technology (ICIT)* (pp. 206-209). IEEE.
27. Saeed Altaee, M. M., & Alanezi, M. (2021). Enhancing cloud computing security by paillier homomorphic encryption. *International Journal of Electrical & Computer Engineering* (2088-8708), 11(2).
28. Parihar, B., Kiran, A., Valaboju, S., Rashid, S. Z., Liyakat, K. K. S., & DR, A. S. L. (2025). Enhancing Data Security in Distributed Systems Using Homomorphic Encryption and Secure Computation Techniques. In *ITM Web of Conferences* (Vol. 76, p. 02010). EDP Sciences.